

MapleStory DX11 2D Engine

언리얼 엔진을 직접 뜯어서 만들어본 프로젝트

C++로 게임을 만들 때 당연히 쓰는 STL, 예외 처리, RTTI 같은 편의 기능을 일부러 안 쓰고, 배열·해시맵·스마트 포인터 같은 기본기를 언리얼 엔진과 같은 원리로 하나하나 직접 만들어본 2D 게임 엔진입니다.

만들려는 게임

메이플스토리 같은 2D 플랫폼어 RPG

사용 언어

C++17 (STL / 예외처리 / RTTI 안 씀)

지금까지 진행률

엔진의 뼈대(기본기) 부분 완료

다음 할 일

화면에 그림을 그리는 렌더러 만들기

배열 / 해시맵 직접 구현

이름을 번호로 바꿔 비교

직접 만든 메모리 관리자

스레드 안전 스마트 포인터

타입 판별 시스템

스킬/버프 시스템

1 한눈에 보는 이 프로젝트

86

자동 테스트 개수

5

성능 개선 완료

3

직접 잡은 실전 버그

6

서로 다른 검증 환경

언리얼이 실제로 쓰는 방식 그대로 만들었어요
해시맵이나 이름 저장 방식, 스마트 포인터 내부 구조를 실제 언리얼 엔진 소스코드와 하나하나 비교해가며 똑같은 원리로 만들었는지 직접 확인했습니다.

윈도우와 리눅스, 두 군데서 다 확인했어요
윈도우에서는 디버그·릴리즈 두 가지로, 리눅스에서는 다른 컴파일러로 세 가지 설정에 메모리 검사 도구까지 돌려서 검증했습니다. 한쪽에서는 안 잡히던 버그가 다른 쪽에서 잡힌 적도 있었어요.

컴파일러가 만든 버그를 직접 찾아냈어요
코드를 세 번이나 고쳐도 크래시가 똑같이 나서, 컴파일러가 제 수정 사항을 실제로는 반영하지 않고 있다는 걸 알아채고 원인을 끝까지 추적했습니다.

특정 상황에서만 터지는 메모리 버그를 고쳤어요
디버그 모드에서는 멀쩡한데 릴리즈 모드에서만 터지는 버그를 원인까지 정확히 찾아서 고쳤습니다.

2 지금까지 만든 것들

분야	이게 뭐고 왜 만들었는지
메모리 관리	게임 전체가 메모리를 요청·반납하는 창구를 하나로 통일했고, 자주 쓰는 작은 크기는 서랍장처럼 미리 나눠뒀서 빠르게 꺼내 쓰게 만들었습니다.
배열 / 해시맵	배열, 셋, 맵처럼 자주 쓰는 자료구조를 직접 만들었고, 특히 해시맵은 데이터를 지웠다 다시 채워도 성능이 안 떨어지게 설계했습니다.
문자열	문자열을 다루는 기본 타입과, "이름"처럼 자주 비교하는 문자열은 한 번만 저장해두고 이후엔 번호로만 비교하게 만들었습니다.
스마트 포인터	객체가 더 이상 안 쓰일 때 자동으로 메모리를 정리해주는 기능이고, 여러 스레드에서 동시에 써도 안전하게 만들었습니다.
오브젝트/타입 시스템	RTTI라는 표준 타입 판별 기능 없이도, 지금 이 객체가 어떤 클래스인지 확인하고 안전하게 형변환하는 시스템을 직접 만들었습니다.
타이머	몇 초 뒤에 한 번 실행하거나, 계속 반복 실행하는 타이머 시스템입니다.
로그 / 오류 검사	개발 중 코드가 잘못됐을 때 알려주는 세 가지 검사 도구와, 로그를 기록하는 시스템을 만들었습니다.
스킬 시스템	버프·디버프, 쿨다운, 도트 데미지(초당 지속 피해)처럼 게임 스킬에 필요한 기능들을 처리하는 시스템입니다.

3 제일 자신 있는 3가지 구현

① 해시맵 — 서랍에 물건 정리하듯 데이터를 저장해요

해시맵은 데이터를 넣을 때 "몇 번 서랍에 넣을지"를 계산해서 그 서랍에 넣는 구조입니다. 같은 서랍에 여러 개가 들어가면 그것들을 사슬처럼 순서대로 연결해두고, 나중에 찾을 때 그 사슬만 따라가면 됩니다. 예전 방식은 데이터를 지울 때 완전히 못 지우고 "치웠음" 표시만 남겨야 했는데, 이 표시가 쌓이면 점점 느려졌습니다. 지금 방식은 지울 때 사슬 연결만 끊으면 끝이라서, 아무리 데이터를 넣었다 뺐다 반복해도 성능이 떨어지지 않습니다.

3번 서랍 —▶ [원소 A] —▶ [원소 B] —▶ (끝)
└─ 같은 서랍에 들어간 원소들이 사슬처럼 줄줄이 연결됨
지울 때: 사슬에서 그 원소만 쏙 빼면 끝 (뒤에 있던 원소들 다 안 밀림)

② 메모리 할당자 — 옷을 크기별로 서랍에 정리해두는 것

작은 메모리를 요청할 때마다 매번 운영체제에 직접 물어보면 느립니다. 그래서 16바이트, 32바이트, 64바이트... 이런 식으로 크기별 서랍을 미리 만들어두고, 필요할 때 그 서랍에서 바로 꺼내 쓰게 만들었습니다. 반납할 때도 그 메모리의 주소만 보고 계산 한 번이면 "이게 몇 번 서랍에서 나온 건지" 바로 알아낼 수 있어서, 반납할 물건마다 꼬리표를 따로 붙여둘 필요가 없습니다.

[큰 페이지 하나] = [정리 노트 | 서랍0 | 서랍1 | 서랍2 | ...]
반납할 때: 주소 계산 한 번 → 정리 노트를 찾음 → 몇 번 서랍인지 읽음 → 그 서랍에 도로 넣음

③ 스마트 포인터 — 마지막으로 나가는 사람이 문을 잠급니다

여러 객체가 서로를 참조하고 있을 때, 정리하는 순서를 잘못 잡으면 같은 메모리를 두 번 지우는 버그가 생길 수 있습니다. 실제로 이 버그를 겪었는데, 표준 라이브러리가 쓰는 방식과 똑같이 해결했습니다. 객체를 관리하는 "관리 장부"를 지울 때, 진짜 객체부터 먼저 지우고 장부는 그다음에 지우도록 순서를 나눴고, 객체를 지우는 도중에 다른 누군가(약한 참조)가 장부를 미리 없애버리지 못하게 자리를 하나 미리 예약했습니다. 그래서 객체 정리가 완전히 끝날 때까지는 장부가 항상 안전하게 살아있습니다.

4 진짜 언리얼이랑 똑같이 만들었는지 하나하나 비교해봤어요

제가 만든 걸로 끝내지 않고, 실제 언리얼 엔진 소스코드를 하나하나 열어서 같은 원리로 만들었는지 직접 대조해봤습니다. 결과는 아래와 같습니다.

만든 것	결과	왜 그런지
배열(TArray)	똑같음	언리얼도 작은 배열은 힙 할당 없이 자기 안에 담아두는 방식을 씁니다. 같은 원리로 만들었어요.
인덱스가 안 변하는 배열	똑같음	지운 자리를 나중에 재사용하는 방식이 언리얼 내부 구조와 똑같아요.
해시맵(Set / Map)	똑같음	사슬로 연결해서 데이터를 찾는 방식이 언리얼이 실제로 쓰는 방식과 같아요.
이름 저장 방식 (FName)	똑같음	이름을 한 번만 저장하고 번호로 비교하는 방식이 언리얼의 핵심 원리와 같아요.
스마트 포인터 내부 구조	똑같음	객체를 관리하는 장부의 구조가 언리얼의 실제 구조와 같아요.
타입 판별 시스템	똑같음	클래스가 뭔지 판별하는 방식이 언리얼이 자동으로 만들어주는 코드의 최소 버전과 같아요.
메모리 할당자	비슷함	핵심 아이디어는 같지만, 언리얼은 스레드별로 훨씬 더 정교하게 처리해요.
스킬 시스템(GAS)	비슷함	개념은 다 똑같이 구현했지만, 온라인 대전용 네트워크 예측 기능은 아직 없어요 (지금은 싱글플레이가 목표라서요).
여러 값 저장하는 맵	다르게 함	언리얼과는 내부 구현 방식이 다른데, 겉으로 동작하는 결과는 똑같아요.

5 가장 고생했던 버그 3가지

① 서로 참조하는 두 객체를 정리할 때 같은 메모리를 두 번 지운 버그

증상	디버그 모드에서는 멀쩡한데, 릴리즈 모드에서만 크래시가 났어요.
원인	두 객체가 서로를 참조하고 있다가 정리될 때, 관리 장부를 너무 일찍 지워버려서 같은 메모리를 두 번 지우고 있었어요. 디버그 모드는 지운 메모리에 특정 값을 채워두는 습관이 있어서 우연히 문제가 가려졌던 거고, 릴리즈 모드는 그 습관이 없어서 바로 드러난 거였어요.
해결	표준 라이브러리가 쓰는 방식 그대로, 자리를 하나 미리 예약해두는 방법으로 순서를 바로잡아서 고쳤어요.

② 컴파일러가 제 수정 사항을 반영 안 하고 있었던 버그

증상	값을 반환하는 아주 짧은 함수 하나에서 크래시가 났는데, 코드를 세 번이나 다르게 고쳐도 크래시가 완전히 똑같이 났어요.
원인	고친 코드가 실제로 실행되고 있지 않다는 뜻이었어요. 컴파일러가 이 짧은 함수를 통째로 다른 코드 안에 끼워 넣어버려서, 함수 안을 아무리 고쳐도 반영이 안 되고 있었던 거예요.
해결	그 함수를 다른 파일로 옮기고 "절대 끼워 넣지 마"라는 옵션을 붙여서, 진짜 별도의 함수 호출로 강제했더니 바로 해결됐어요.

③ 검사용 코드 안에 진짜 동작을 같이 넣어버린 버그

증상	릴리즈 모드에서만 빈 배열에 접근하다가 크래시가 났어요.
원인	검사용 코드(check) 안에 "배열에 값을 추가하는" 실제 동작까지 같이 넣어버렸는데, 릴리즈 모드에서는 이 검사용 코드 자체가 통째로 사라지는 규칙이 있어서 추가하는 동작도 같이 사라져버렸어요. 그래서 배열이 텅 빈 채로 다음 코드가 실행된 거였어요.
해결	동작하는 코드와 검사하는 코드를 무조건 분리하기로 규칙을 정했고, 이후로는 릴리즈 모드와 똑같은 조건으로도 항상 미리 테스트하게 만들었어요.

6 어떻게 검증했는지

- 테스트 86개를 만들어서 매번 확인했어요 — 각 기능을 만들 때마다 테스트를 미리 써놓고, 이 테스트를 전부 통과해야만 다음 단계로 넘어가는 규칙을 지켰어요. 데이터를 넣었다 뺐다 반복하는 테스트, 여러 스레드가 동시에 접근하는 테스트도 포함돼요.
- 윈도우에서 디버그·릴리즈 두 가지 다 확인했어요 — 위에서 말씀드린 것처럼 릴리즈 모드에서만 나는 버그가 있었기 때문에, 이후로는 항상 두 가지 모드를 다 확인하게 됐어요.
- 리눅스에서도 다른 컴파일러로 한 번 더 확인했어요 — 윈도우 컴파일러와는 다른 컴파일러(g++), 그 리고 메모리 검사 도구(valgrind)로 다시 확인해서, 한쪽에서 못 잡는 문제를 다른 쪽에서 잡을 수 있게 했어요.

7 앞으로 할 일

- ✓ 엔진의 기본기 — 메모리 관리, 배열/해시맵, 문자열, 스마트 포인터, 오브젝트 시스템, 타이머, 스킴 시스템까지 다 만들었어요
- ✓ 성능 개선 작업 5가지도 다 끝냈어요 — 메모리 할당자, 인라인 배열, 해시맵 구조, 이름 저장 방식, 스마트 포인터 스레드 안전화
- ▶ 다음은 화면에 그림을 그리는 렌더러예요 — 캐릭터, 배경, 카메라 같은 걸 실제로 화면에 보여주는 작업입니다 (**DirectX11** 사용)
 - 그다음은 애니메이션, 물리(충돌 판정), 사운드, UI, 조작, 리소스 로딩, 맵 로딩이에요
 - 마지막은 진짜 게임플레이예요 — 캐릭터, 스킴, 몬스터 AI, 인벤토리, 퀘스트, 저장 기능

MapleStory DX11 2D Engine · STL·예외처리·RTTI 없이 C++17로 언리얼 엔진의 구조를 직접 재현한 프로젝트 · 더 자세한 구현 원리는 ARCHITECTURE.md, 문제 해결 과정 전체 기록은 PROJECT_SUMMARY.md 참고